1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

# 🚦 Algorithm Flowcharts

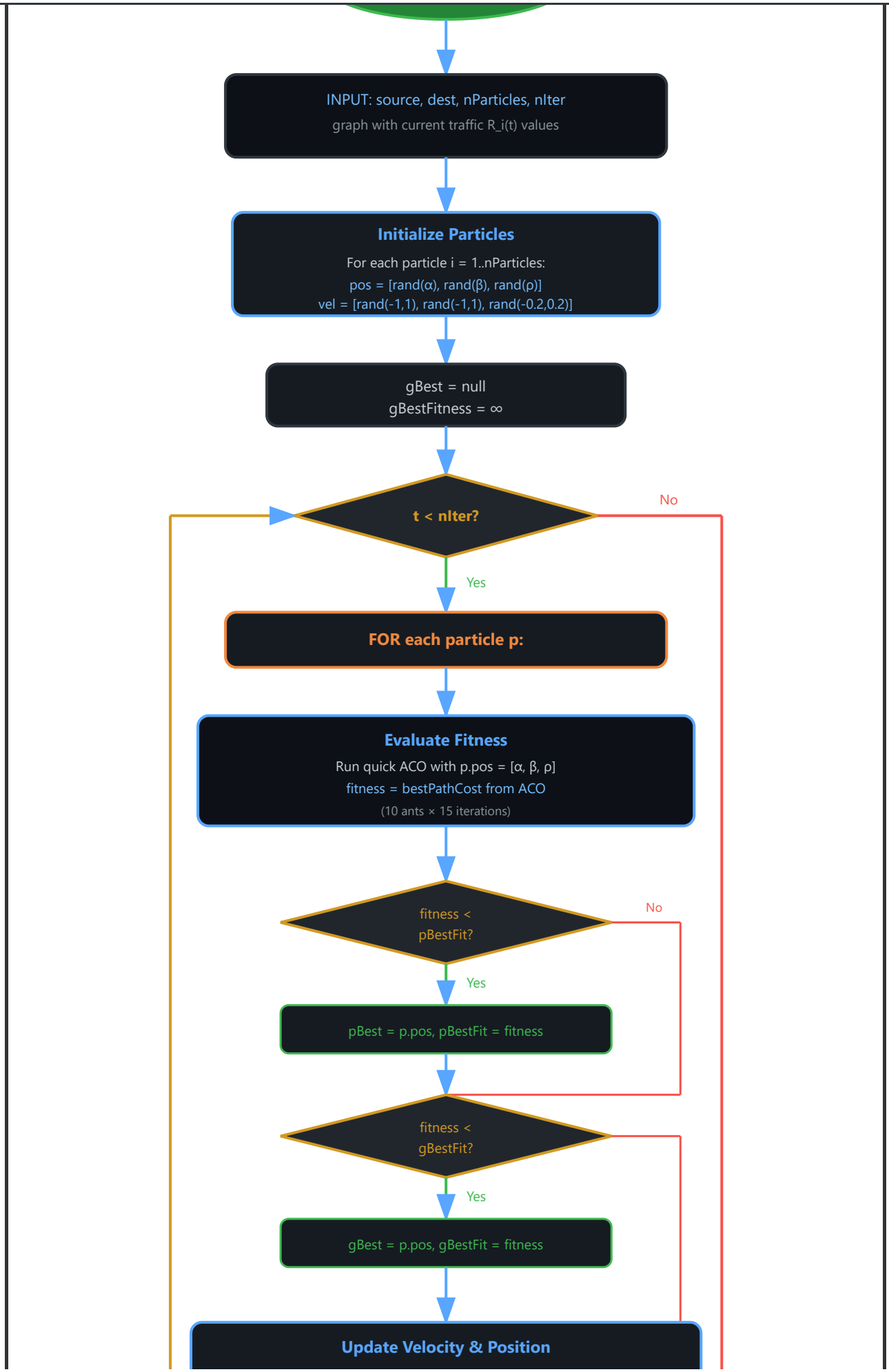
PSO-ACO Hybrid Dynamic Path Planner for Dhaka City — Based on index.html Implementation

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

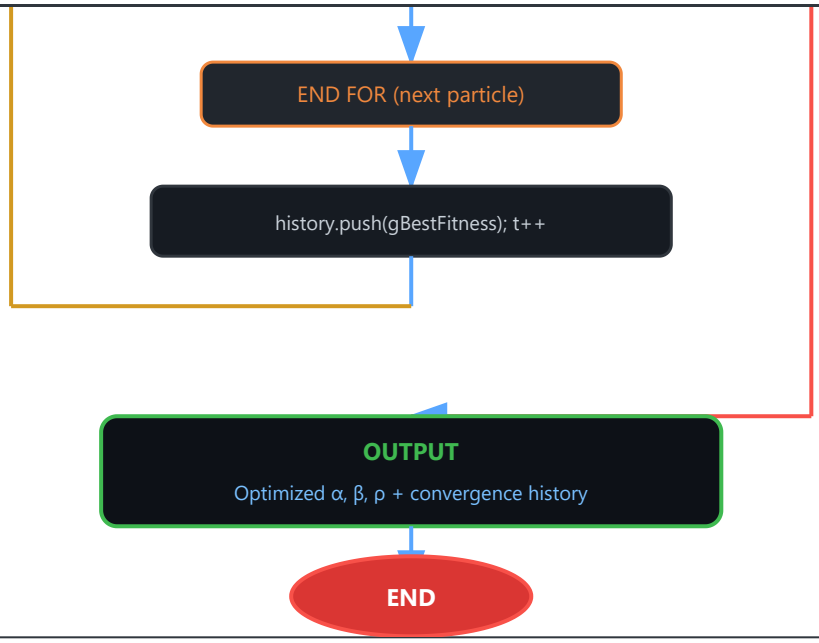
# 1 PSO Optimization (Parameter Tuning)

Particle Swarm Optimization to find optimal α, β, ρ for ACO

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation



1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation




Key Formulas Used (from code – PSOOptimizer class):

$v[d] = 0.72 \times v[d] + 1.49 \times r_1 \times (pBest[d] - pos[d]) + 1.49 \times r_2 \times (gBest[d] - pos[d])$

$pos[d] = pos[d] + v[d]$

Bounds: $\alpha \in [0.5, 5.0]$, $\beta \in [0.5, 5.0]$, $\rho \in [0.05, 0.95]$

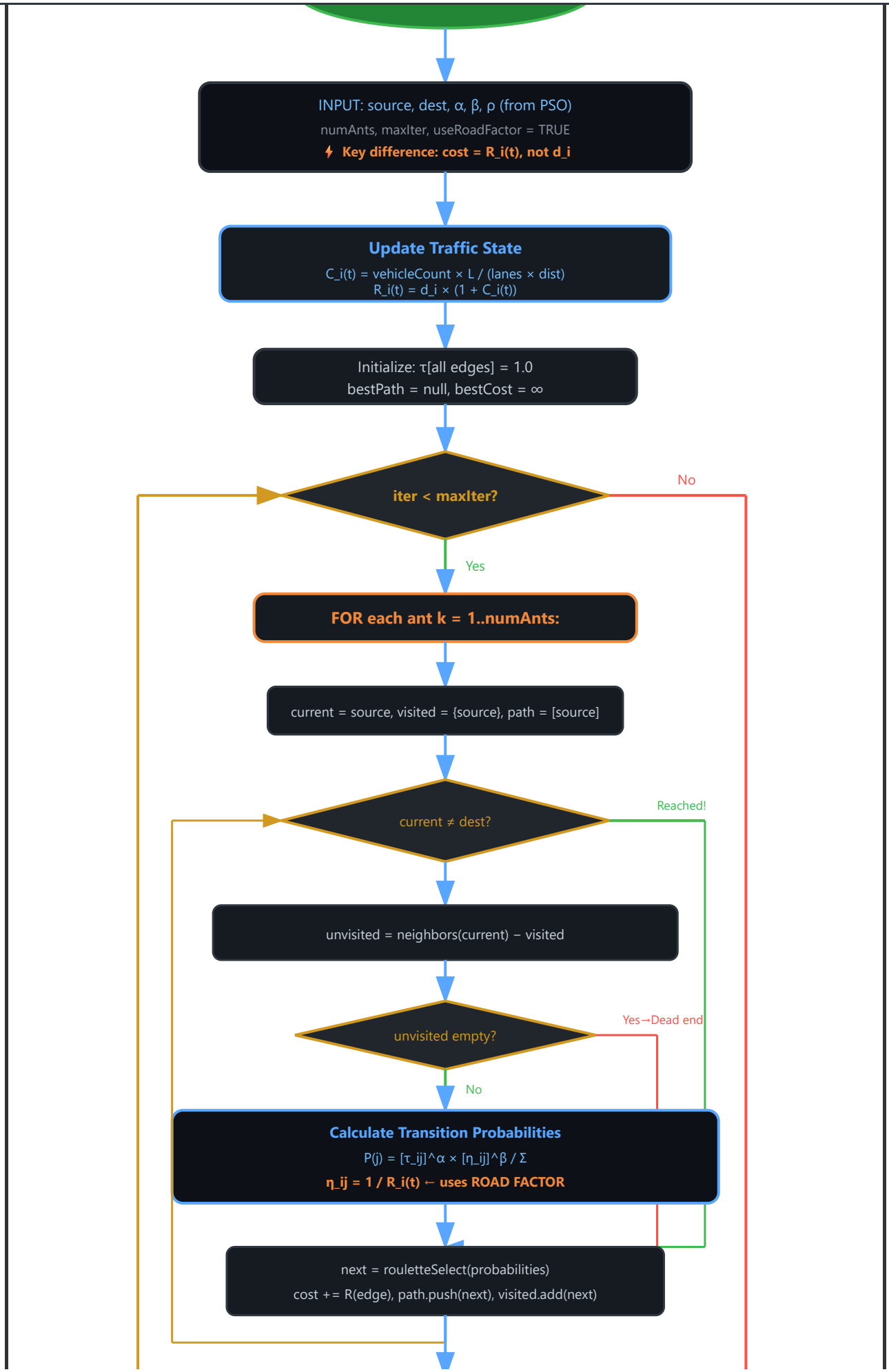
fitness(α,β,ρ) = bestPathCost from ACO(10 ants, 15 iters, useRoadFactor=true)

💡 *Right-click → "Save Image As" or use Print (Ctrl+P) to save as PDF*

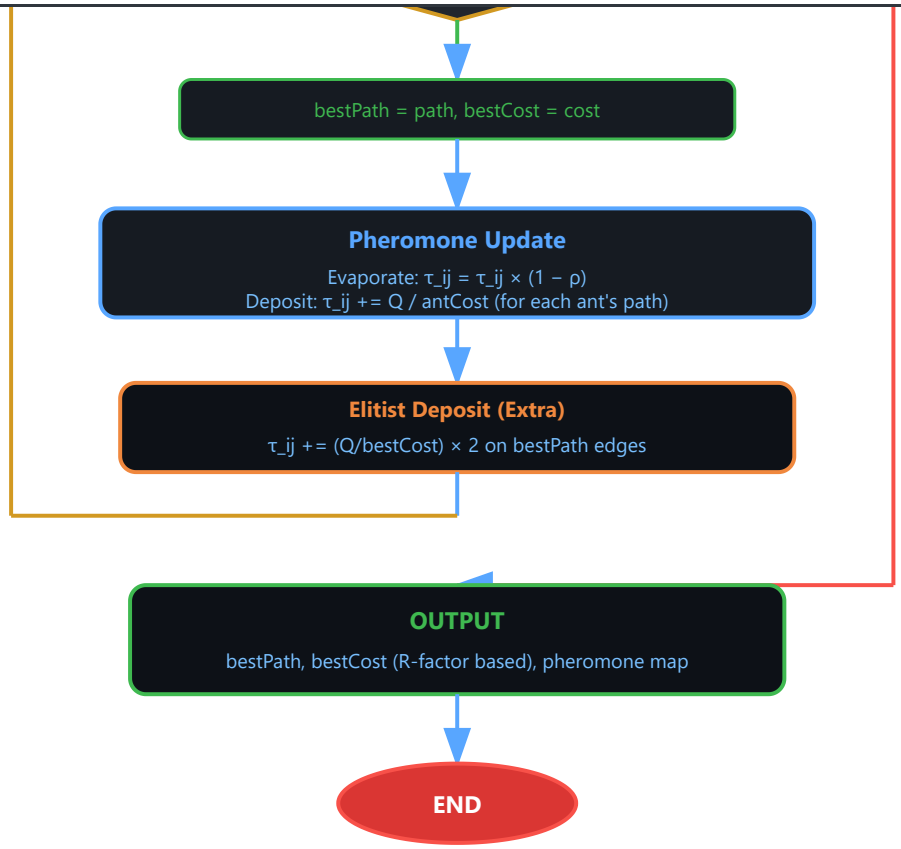
## 2 Improved ACO (Road Condition Factor)

Ant Colony Optimization using $R_i(t) = d_i \times (1 + C_i)$ as edge cost

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation



1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

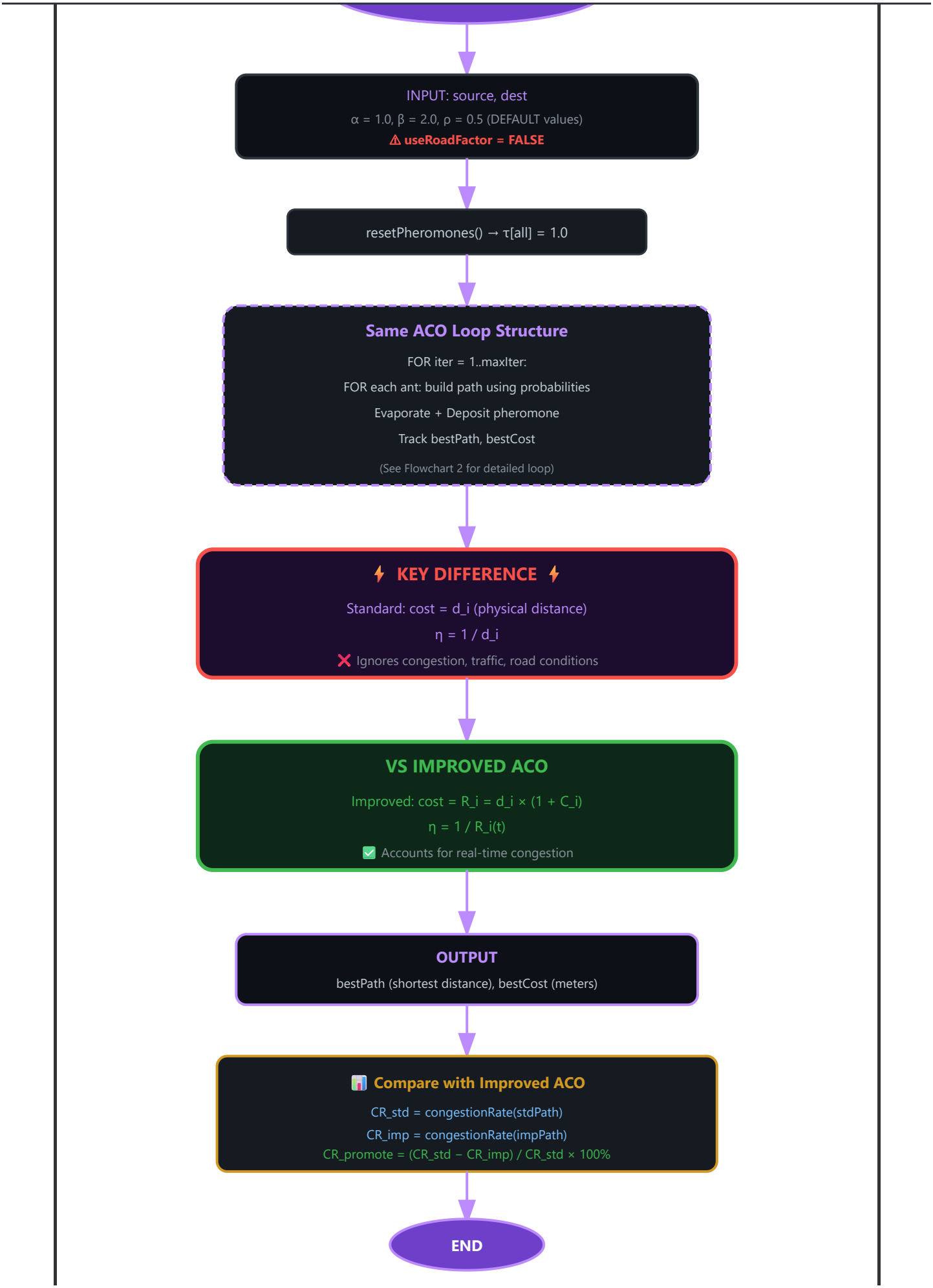



```
Key Formulas (ACOPathfinder class, useRoadFactor = true):

C_i(t) = (vehicleCount × 4.5) / (lanes × distance)
R_i(t) = d_i × (1 + C_i(t)) ← ROAD CONDITION FACTOR
η_ij = 1 / R_i(t) ← heuristic uses R, not d
P_ij = [τ_ij]^α × [η_ij]^β / Σ ← transition probability
Evaporation: τ = τ × (1 − ρ), min = 0.001
Deposit: Δτ = Q / pathCost
Elitist: Δτ_best = (Q / bestCost) × 2
```

## 3 Standard ACO (Distance-Based Baseline)

Same ACO algorithm but using physical distance d_i instead of road factor R_i

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation



1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

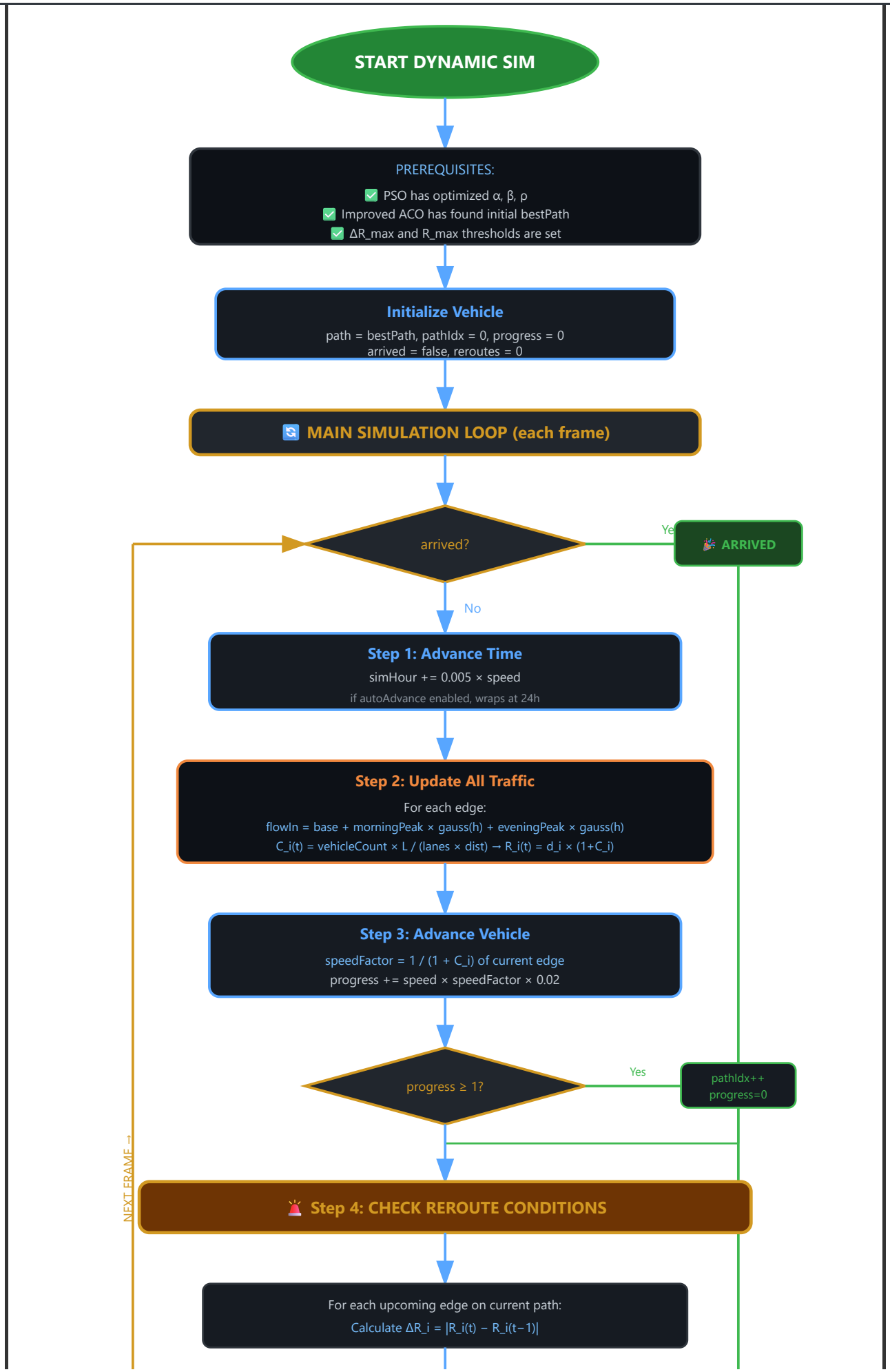
```
Standard: edgeCost = d_i (distance in meters)
Improved: edgeCost = R_i(t) = d_i × (1 + C_i(t))

CR = Σ(R_i for congested edges) / Σ(R_i for all edges)
where "congested" = C_i > 0.3

CR_promote = (CR_standard − CR_improved) / CR_standard × 100%
Positive value = improved method avoids more congested roads ✅
```

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

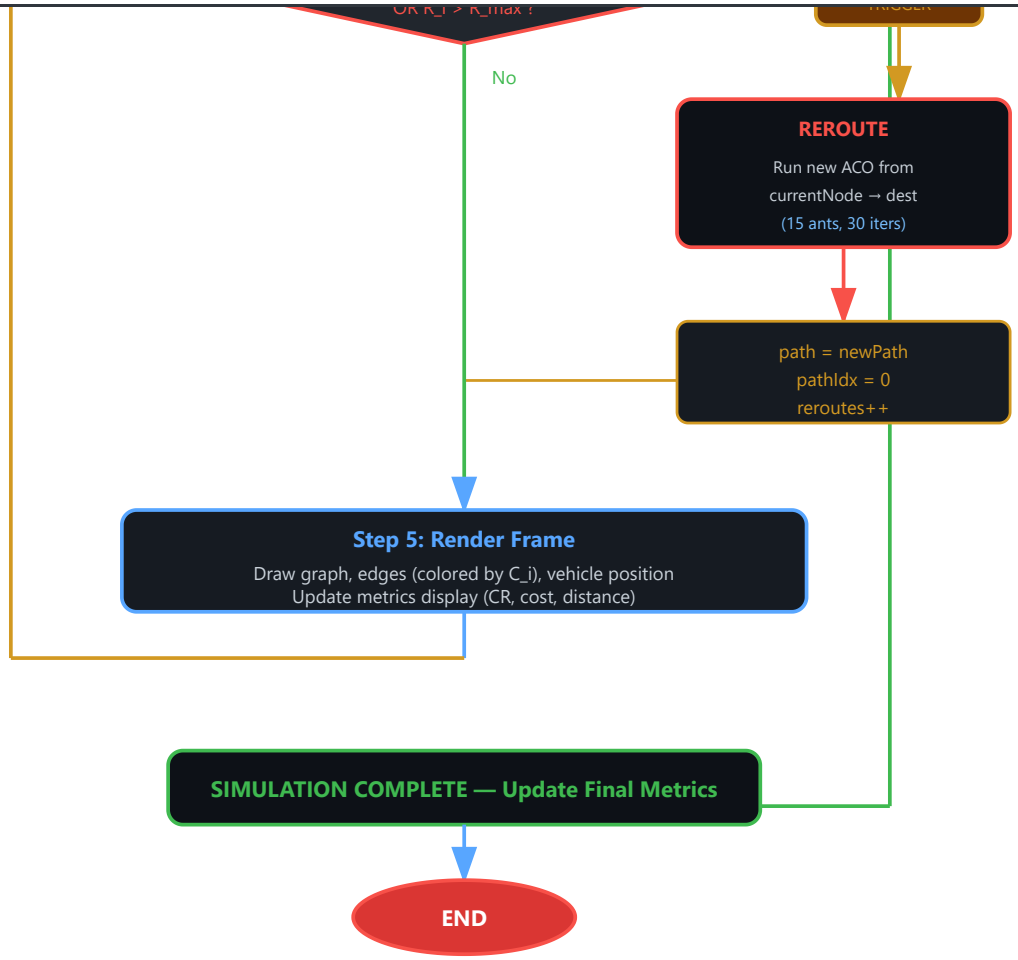
## 4️⃣ Dynamic Simulation (Real-time Rerouting)

Vehicle moves along path, traffic updates each tick, reroutes when congestion exceeds thresholds

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation



1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

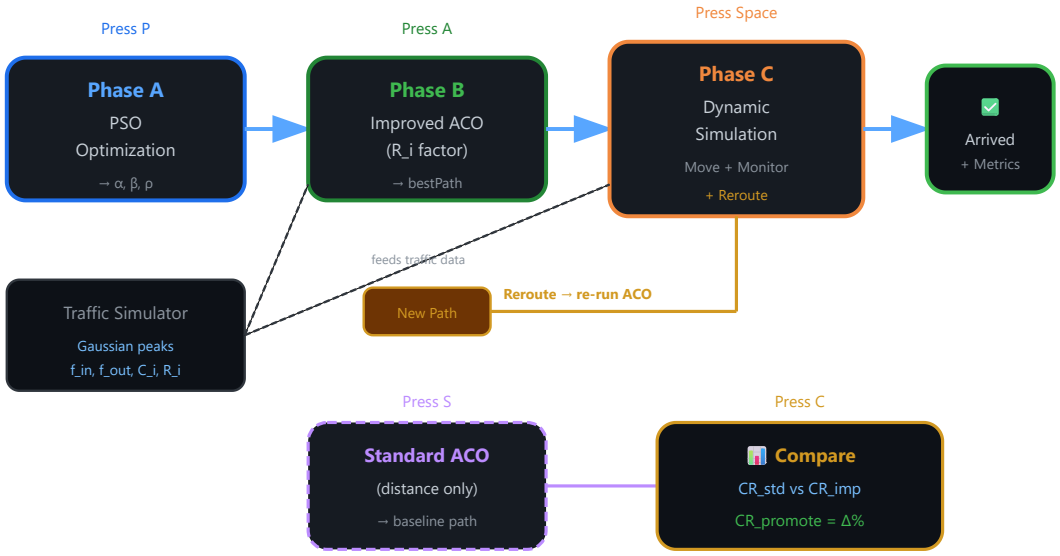



```
Key Dynamic Rerouting Logic (DynamicPlanner class):

Vehicle speed on edge: actualSpeed = speed × 1/(1 + C_i)
→ Higher congestion = slower movement (realistic!)

REROUTE TRIGGER CONDITIONS (checked every 30 frames):
  Condition 1: ΔR_i = |R_i(t) − R_i(t−1)| > ΔR_max (sudden
change)
  Condition 2: R_i(t) > R_max (absolute congestion limit)

If triggered: Run new ACO from currentNode → destination
  Using PSO-optimized params, 15 ants, 30 iterations
  New path replaces old path, reroute counter increments
```

1. PSO Optimization | 2. Improved ACO | 3. Standard ACO | 4. Dynamic Simulation

## 🔗 Overall System Pipeline

How all four components connect in the index.html implementation




Algorithm Complexity: $O(n \times (n-1)^2 \times m \times T/2)$ where n=nodes, m=ants, T=iterations

📄 All flowcharts based on the `index.html` implementation
Paper: "Dynamic Path Planning Based on Improved ACO in Traffic Congestion" — Wu et al.

💡 **To save as images:** Use browser Print (Ctrl+P → PDF), or right-click → screenshot, or use a browser extension